MFE Python Course

Productionization, Part 1

Course Progress Overview

- ✅ learned python
- ✅ built features
- ✅ trained model
- [ ] deployed model

Today we bridge the gap between research and production code

Key transition: Moving from exploratory analysis to reliable systems

Final assignment preview: Building a production-grade model serving endpoint

Agenda

1. Debugging
2. Linting
3. Testing
4. Web frameworks
5. Containers

The Cost of Poor Production Practices

Knight Capital's $440M Loss in 45 Minutes

- Aug 1, 2012: New trading algorithm deployed to production
- Code reused old "Power Peg" flag accidentally left in codebase
- Deployment skipped testing on 1 of 8 servers
- Result: 4 million unintended trades in 45 minutes

Key Lessons:
- Seemingly minor deployment issues can cascade catastrophically
- Missing tests and quality checks have real financial impact
- Production systems need rigorous deployment processes
- **Financial markets punish technical mistakes severely**

From Research to Production

Jupyter Notebooks: Great for Research, Risky for Production

Benefits of Notebooks:
- Quick experimentation and visualization
- Interactive debugging and data exploration
- Great for presentations and documentation
- Easy to share research findings

Why They're Not Production-Ready:
- Hidden state and non-linear execution
- Poor version control and code review
- No proper error handling or logging
- Hard to test and maintain

Production Code Requirements:
- Modular, reusable functions
- Proper exception handling
- Automated testing
- Consistent style and documentation
- Reproducible builds

Core Requirements for Production ML

Reliability
- Consistent model performance
- Graceful error handling
- Data validation

Scalability
- Handle varying load
- Resource efficiency
- Quick deployment

Maintainability
- Clear code structure
- Documentation
- Version control

Monitoring
- Performance metrics
- Model drift detection
- System health

Debugging Fundamentals

The Purpose of Debugging

Beyond Just Fixing Bugs:
- Understanding code behavior is as important as fixing issues
- Debugging is a crucial skill for technical interviews

Proactive Debugging:
- Code reviews and testing catch issues early
- Logging and monitoring prevent production failures

Reactive Debugging:
- Systematic approach to solving production issues
- Focus on root causes, not just symptoms

Common ML Debugging Scenarios

Data Pipeline Issues:
- Missing or corrupted data
- Incorrect preprocessing steps
- Data type mismatches
- Unexpected outliers/nulls

Model Behavior:
- Prediction anomalies
- Performance degradation
- Training instability
- Resource usage spikes

Common in Finance:
- Market data inconsistencies
  - Stale prices in real-time feeds
  - Incorrect adjustment for corporate actions
  - Missing dividend data affecting returns

- Risk calculation errors
  - VaR calculations using wrong time window
  - Greeks computed with stale volatility surface
  - Correlation matrix not positive definite

- Portfolio rebalancing issues
  - Position reconciliation mismatches
  - Wrong FX rates for international securities
  - Trading volume constraints violated

- Time-sensitive failures
  - Market open/close timing bugs
  - Options expiry date calculations
  - Settlement date processing errors

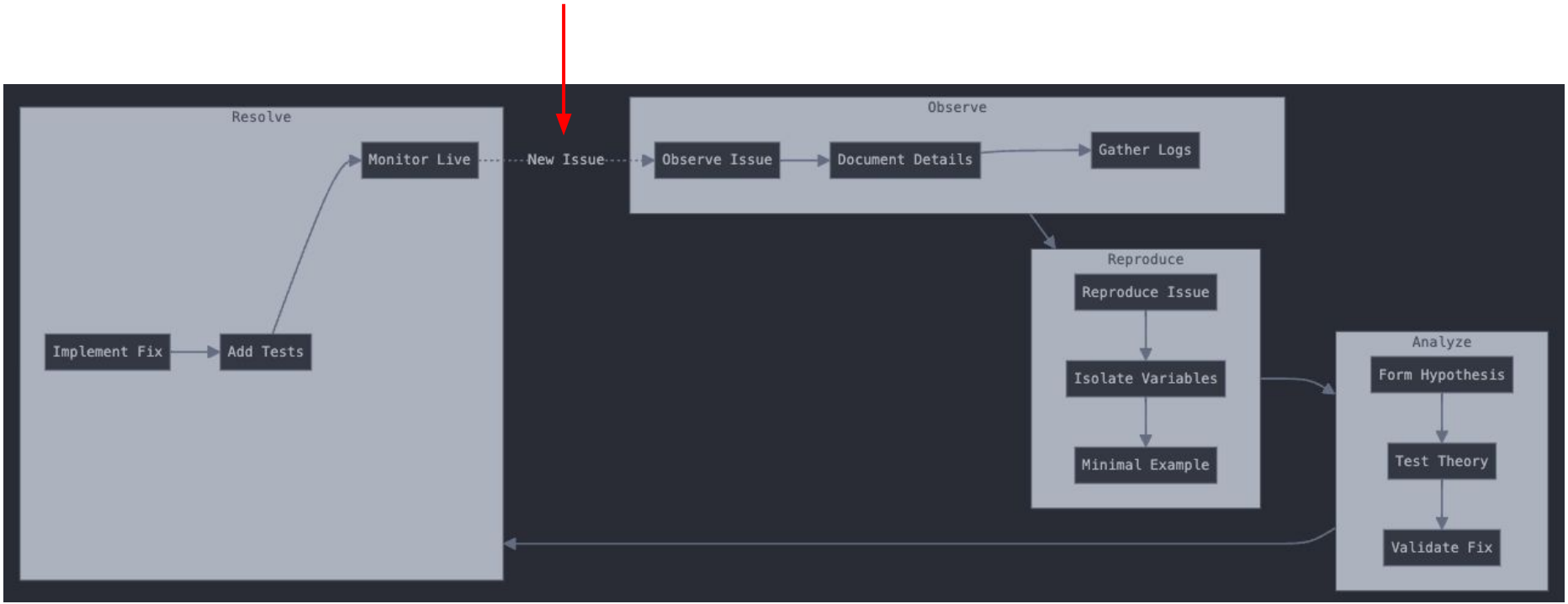
Key Steps in Systematic Debugging

1. Observe & Document
   - Log exact error messages
   - Note system state
   - Record reproduction steps

2. Reproduce & Isolate
   - Create minimal example
   - Control variables
   - Identify patterns

3. Hypothesize & Test
   - Form theories
   - Write targeted tests
   - Validate assumptions

4. Fix & Verify
   - Implement solution
   - Add regression tests
   - Monitor in production


Resolve
Monitor Live
New Issue
Observe
Observe Issue
Document Details
Gather Logs
Reproduce
Reproduce Issue
Isolate Variables
Minimal Example
Analyze
Form Hypothesis
Test Theory
Validate Fix
Implement Fix
Add Tests

Debugging Tools

1. Print Statements
- Quick but messy
- Limited context
- Hard to maintain



2. Logging
- Structured output
- Different log levels
- Persistent records



3. Interactive Debuggers
- Step-through execution
- Variable inspection
- Breakpoint control



4. Modern Tools
- IDE integration
- Performance profilers
- Memory analyzers

Code Quality & Linting

Why Code Quality Matters

Cost of Poor Quality
- 42% of developer time spent maintaining legacy code
- 2.5x longer to onboard new team members
- Higher risk of production incidents

Financial Industry Stakes
- Regulatory compliance requirements
- Audit trail importance
- Multi-team code dependencies
- High cost of errors

Team Benefits
- Faster code reviews
- Easier debugging
- Better knowledge sharing
- Reduced technical debt

What is Linting

Definition
- Automated code analysis tool
- Flags programming and stylistic errors
- Enforces consistent coding standards
- Named after Unix "lint" tool

What Linters Check:
- Syntax errors
- Variable naming
- Code complexity
- Unused imports
- Style guide violations

Why We Lint:
- Catch errors before runtime
- Enforce team standards automatically
- Reduce code review friction
- Improve code maintainability

When to Lint:
- During development (IDE integration)
- Pre-commit hooks
- Continuous Integration
- Before code review

Linting Tools - Essential Stack

flake8
- Style guide enforcement
- Syntax error detection
- Complexity checks
- Custom rule support

black
- Automatic code formatting
- Non-negotiable style
- Git-friendly
- Team consistency

isort
- Import organization
- Module grouping
- Dependency clarity
- Standardized layout

mypy
- Static type checking
- Runtime error prevention
- Better IDE support
- Self-documenting code

Testing

Why We Test

Testing in Financial Systems:

1. Risk Mitigation
- Prevent costly production errors
- Validate business logic
- Ensure numerical accuracy

2. Quality Assurance
- Maintain code reliability
- Verify model behavior
- Protect critical functionality

3. Team Confidence
- Safe refactoring
- Faster deployments
- Clear documentation

Expectations of Quants:

- Write tests before implementing complex financial calculations

- Validate against known edge cases (e.g., market stress scenarios)

- Document test cases with clear financial interpretation

- Test numerical precision and floating-point behavior

- Verify model behavior across different market regimes

- Include regulatory compliance checks in test suites

Testing Throughout the Stack

1. Local Development
- Unit tests
- Quick feedback
- Development validation

2. Continuous Integration
- Full test suite
- Integration checks
- Performance tests

3. Staging
- Production-like testing
- Load testing
- Data validation

4. Production
- Smoke tests
- Monitoring
- A/B testing

Testing Throughout the Stack

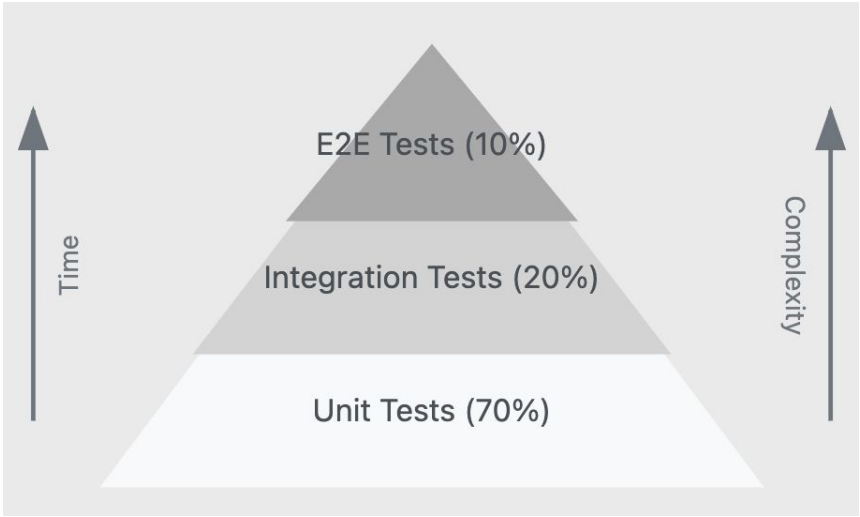
Test Distribution Explained:

- 70% Unit Tests: The majority of your tests should be unit tests that verify individual components in isolation. These are fast, reliable, and help pinpoint issues quickly.

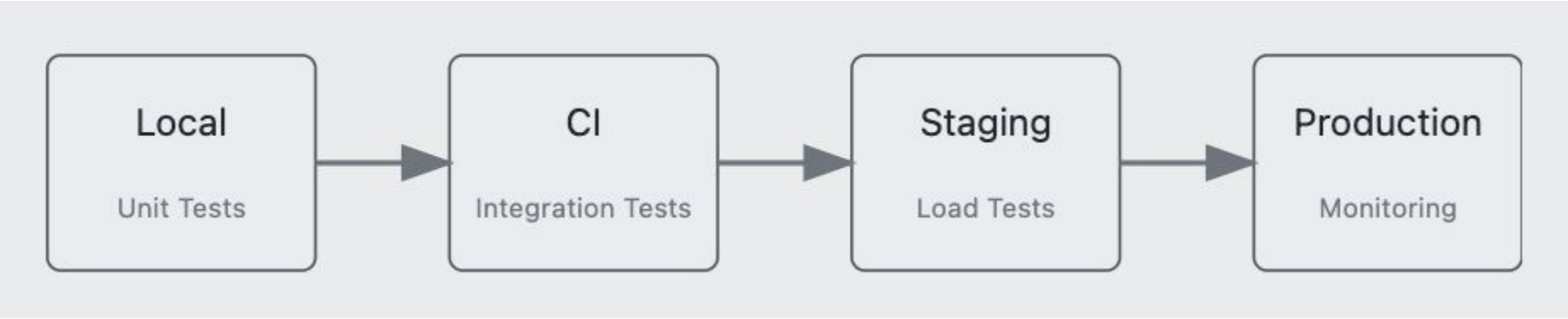
- 20% Integration Tests: A smaller portion should test how components work together, like checking if your data pipeline properly feeds into your model.

- 10% End-to-End Tests: The smallest portion should be comprehensive tests that verify entire workflows, as these are the most time-consuming and complex to maintain.

This ratio helps teams balance between comprehensive testing and development velocity.



Testing Throughout the Stack



Stages of Testing

Testing ML Components:

Data Preprocessing
- Input validation
- Feature engineering
- Data transformations
- Edge cases

Model Components
- Layer configurations
- Custom loss functions
- Optimization settings
- Numerical stability

Prediction Logic
- Output formats
- Inference pipeline
- Performance constraints
- Error handling

Testing System Interactions:

Data Pipeline Integration
- Data source connections
- ETL processes
- Data quality checks
- Storage systems

Model Training Pipeline
- Feature pipeline
- Training workflow
- Model serialization
- Version control

Inference Pipeline
- API endpoints
- Batch processing
- Real-time scoring
- Error recovery

Full System Validation:

System Tests
- Complete workflows
- Business requirements
- Cross-component interaction
- Error scenarios

Performance Tests
- Response times
- Resource usage
- Concurrent users
- Data throughput

Load Tests
- Peak capacity
- Failure points
- Recovery behavior
- SLA validation

Unit tests (local)

Integration tests (CI/CD)

End-to-end tests (staging)

Unit Test Fundamentals

Test Coverage
Your tests should cover all critical paths through your code, not just the happy path. Aim for at least 80% coverage in financial applications, with complete coverage of risk calculations and trade processing logic. Coverage tools help identify untested code, but focusing solely on coverage metrics can be misleading.

Test Reproducibility
Tests must produce the same results every time they run, regardless of environment or timing. Seed random number generators, mock time-dependent functions, and avoid dependencies on external state. This is especially crucial for financial models that use stochastic processes or market data.

Test Independence
Each test should be completely isolated and self-contained. Tests should never depend on the results of other tests or share mutable state. Reset any shared resources (like database connections or global variables) between test runs to prevent cross-contamination.

Unit Test Fundamentals

Test Determinism
Tests should never have timing dependencies or race conditions. When testing time-sensitive operations like options pricing or settlement calculations, explicitly control the time context rather than relying on the system clock.

Positive vs Negative Testing
Positive Tests: Verify that your code works correctly under expected conditions (e.g., calculating Greeks for standard option contracts).
Negative Tests: Ensure your code fails gracefully under invalid conditions (e.g., handling malformed trade data or invalid market quotes).

Test Isolation
Mock external dependencies (APIs, databases, file systems) to ensure tests only validate the specific unit of code being tested. For financial applications, this often means creating synthetic market data and mock trading systems.

Pytest Fundamentals and Industry Standards

Test Structure
Each unit test should focus on a single functionality and follow the Arrange-Act-Assert pattern: set up the test conditions, execute the function under test, and verify the results match expectations. Break complex tests into multiple focused test cases.

Naming and Organization
Structure test files to mirror your production code's organization, prefixing files with 'test_'. Name test functions descriptively to clearly indicate the scenario being tested, such as 'test_portfolio_rebalance_handles_negative_weights'.

Test Data Management
Create reusable test fixtures using pytest.fixture to maintain consistent test data across multiple test cases. For financial data, include edge cases like market holidays, corporate actions, and extreme market conditions in your test datasets.

Assertions and Validation
Use pytest's built-in assert statements along with numpy.testing for numerical comparisons, being mindful of floating-point precision in financial calculations. Include appropriate tolerance levels when comparing floating-point results.

Pytest Fundamentals and Industry Standards

Mocking and Dependencies
Utilize pytest's monkeypatch or unittest.mock to isolate the code under test from external dependencies like market data feeds or databases. Create mock objects that accurately represent the behavior of your production systems.

Error Testing
Explicitly test error conditions using pytest.raises to verify that your code handles exceptions appropriately, especially for critical scenarios like invalid trades or data inconsistencies that could lead to financial losses.

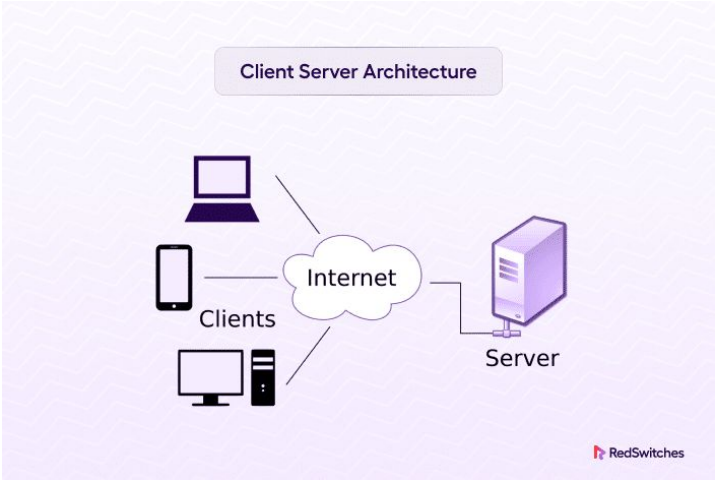
Documentation and Maintenance
Include docstrings in test functions to explain the business context and importance of each test case. Regularly review and update tests when business requirements or market conditions change.

Web Frameworks

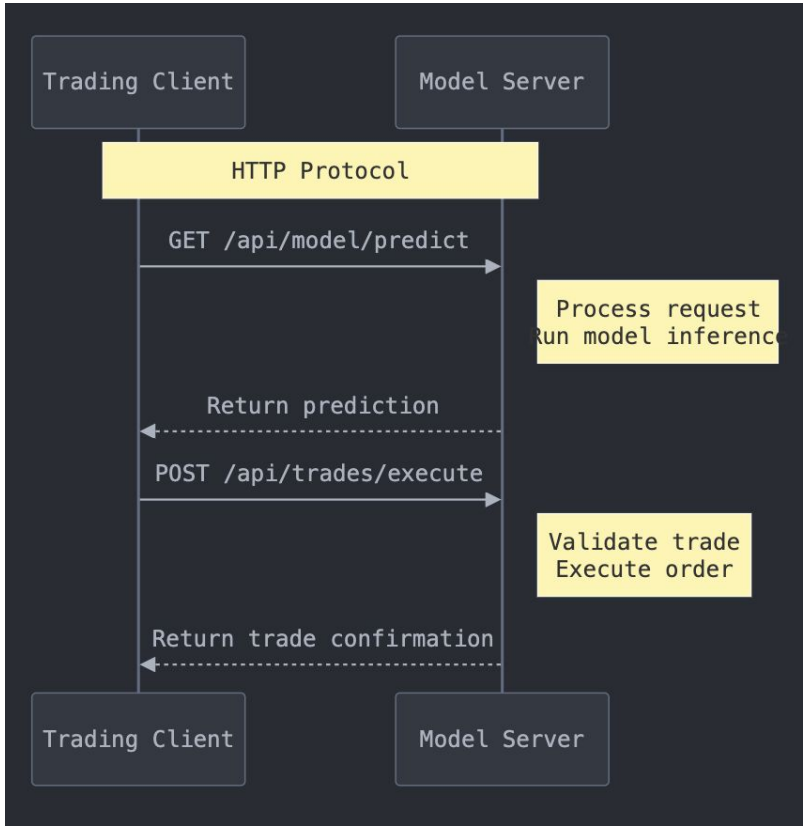
Web Services Fundamentals

Client-Server Basics
- Client: Makes requests (e.g., trading interface)
- Server: Processes and responds (e.g., model service)
- Protocol: HTTP defines how they communicate
- API: Interface for programmatic interaction

Key Advantages
- Scalable architecture
- Language-agnostic
- Microservice friendly
- Easy to monitor



Client-Server Architecture



REST APIs

REST Architecture
- Representational State Transfer
- Standard way to structure web APIs
- Resources identified by URLs
- Uses HTTP methods meaningfully

Best Practices
- Use nouns for resources (/trades not /execute-trade)
- Versioning in URL (/v1/trades)
- Clear status codes (200 OK, 400 Bad Request)
- Consistent error formats

HTTP Methods

GET: Retrieve data (e.g., get portfolio value)

POST: Create new resource (e.g., submit trade)

PUT: Update existing resource (e.g., modify order)

DELETE: Remove resource (e.g., cancel trade)

REST APIs

HTTP Methods

GET: Retrieve data (e.g., get portfolio value)

POST: Create new resource (e.g., submit trade)

PUT: Update existing resource (e.g., modify order)

DELETE: Remove resource (e.g., cancel trade)



CRUD Methods

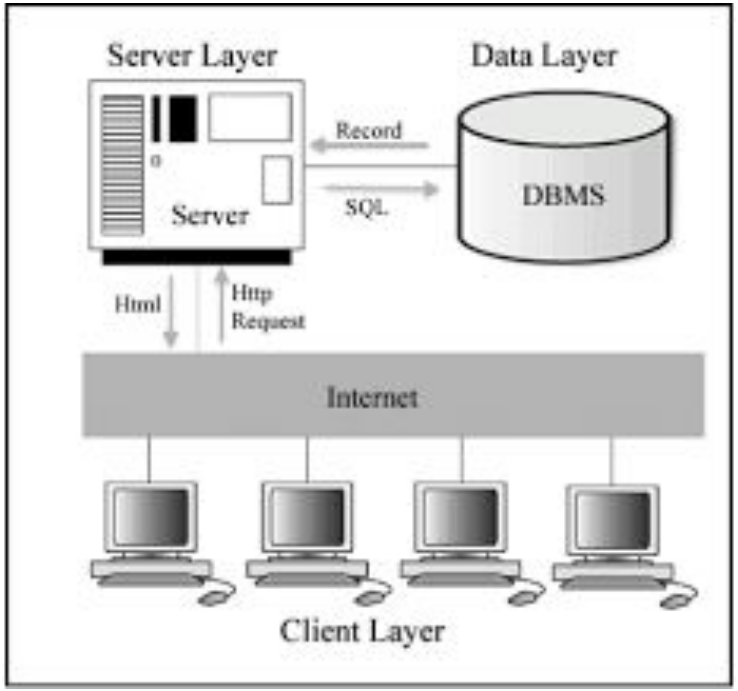
CREATE: Create new resource (e.g., submit trade)

RETRIEVE: Retrieve data (e.g., get portfolio value)

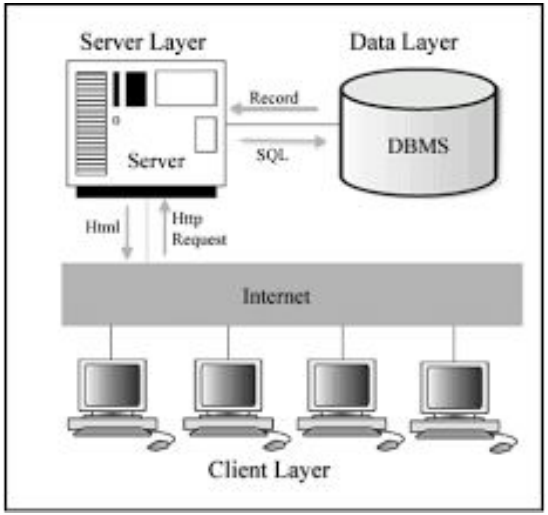
UPDATE: Update existing resource (e.g., modify order)

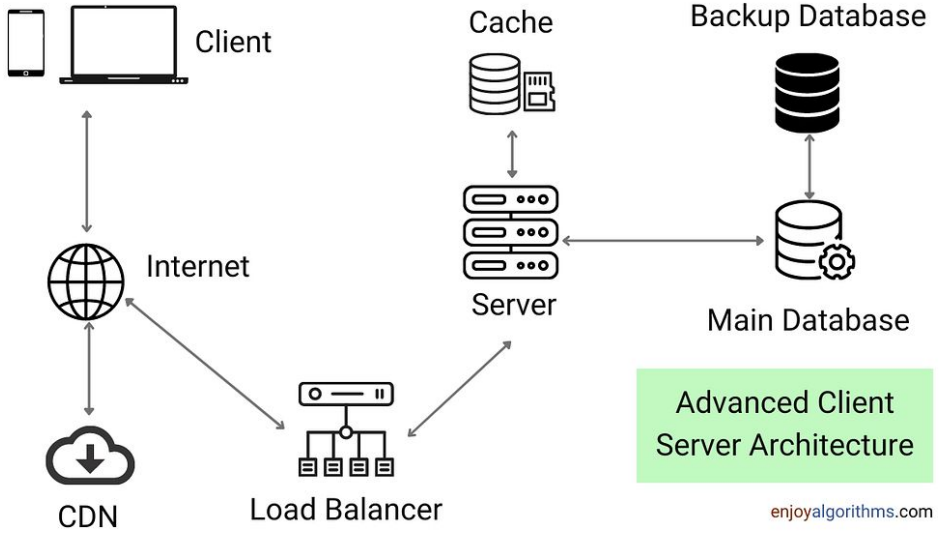
DELETE: Remove resource (e.g., cancel trade)

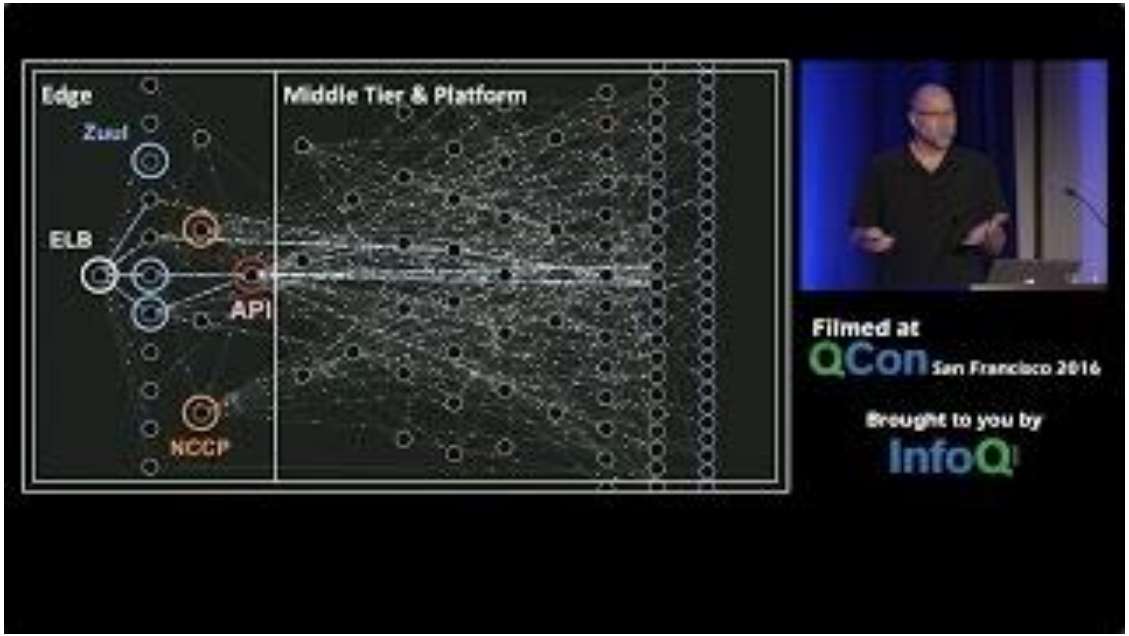
Client-Server Architecture



Client-Server Architecture






Edge
Middle Tier & Platform
Zuul
ELB
API
NCCP
Filmed at
QCon San Francisco 2016
Brought to you by
InfoQ

FastAPI - Python Web Framework

Advantages for Quant Applications

Speed
- High-performance ASGI framework
- Async support for concurrent requests
- Critical for real-time trading systems

Developer Experience
- Modern Python features (type hints)
- Automatic OpenAPI documentation
- Built-in request validation
- IDE support with type checking

ML-Friendly
- Native Pydantic integration
- Easy numpy/pandas handling
- Background task support
- Efficient binary data handling

Request-Response Cycle

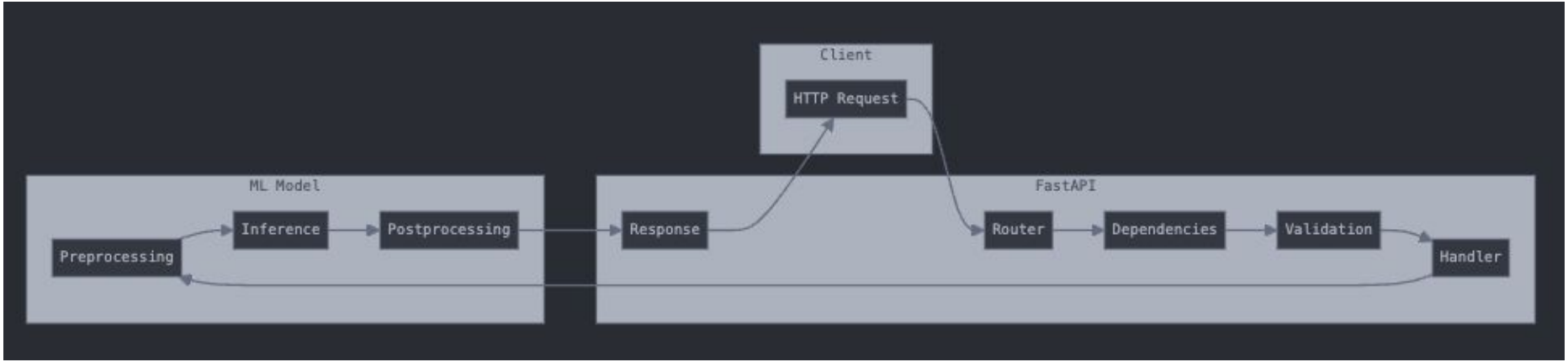
1. Request Validation
- Type checking of inputs
- Data format verification
- Business rule validation
- Security checks

2. Data Processing
- Input normalization
- Feature engineering
- Batch processing
- Async operations

3. Response Formatting
- Consistent JSON structure
- Error standardization
- Status codes
- Performance metrics

4. Error Handling
- Graceful degradation
- Informative messages
- Proper logging
- Monitoring alerts

Request-Response Cycle



FastAPI Best Practices

Type Safety
- Use Pydantic models for all I/O
- Define clear request/response schemas
- Leverage type hints throughout
- Enable mypy checking

Path Operations
- Organize routes logically
- Use appropriate HTTP methods
- Include response models
- Document edge cases

Request Handling
- Validate all inputs
- Use dependency injection
- Handle timeouts properly
- Rate limit when needed

Model Serving
- Load models efficiently
- Cache when appropriate
- Monitor inference times
- Version control models

FastAPI Best Practices

Without Dependency Injection:

python

```python
from fastapi import FastAPI

app = FastAPI()

def get_database_connection():
    # Imagine this creates a new database connection
    return "database_connection"

@app.get("/items/")
def read_items():
    db = get_database_connection()  # Directly creates a connection
    return {"db": db}
```

FastAPI Best Practices

With Dependency Injection:

python

```python
from fastapi import FastAPI, Depends

app = FastAPI()

# Dependency to provide the database connection
def get_database_connection():
    return "database_connection"

@app.get("/items/")
def read_items(db=Depends(get_database_connection)):
    return {"db": db}
```

ML Model Serving - Deployment Considerations

Model Management
- Version control of models
- A/B testing support
- Model warmup
- Resource optimization

Request Processing
- Input validation
- Batch inference
- Timeout handling
- Error recovery

System Design
- Horizontal scaling
- Load balancing
- Health checks
- Performance monitoring

Common Pitfalls
- Memory leaks from large models
- Slow cold starts
- GIL limitations
- Resource contention

Containers

Why Containers

Development Benefits
- Consistent environments across team
- "Works on my machine" problem solved
- Fast onboarding of new team members
- Easy local testing of production code

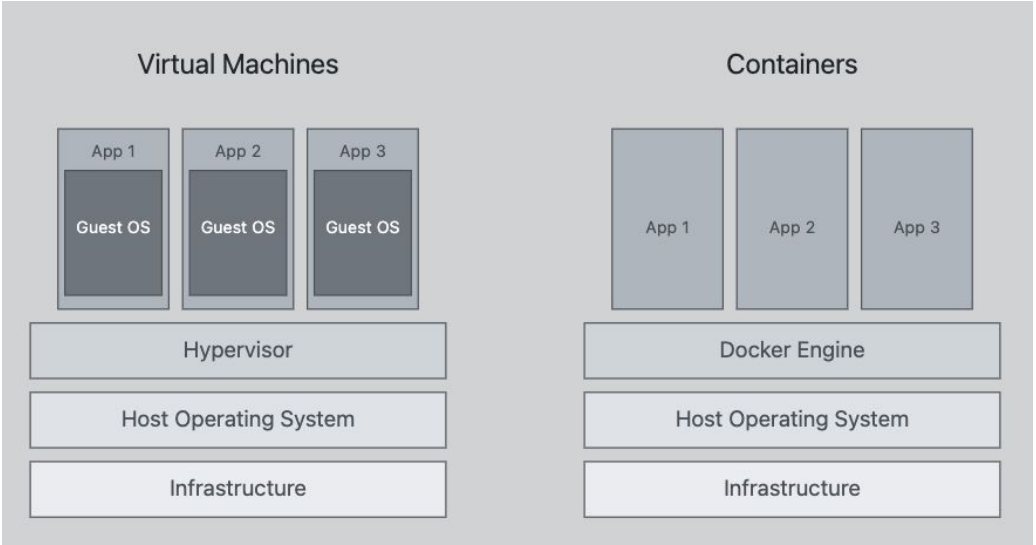
Operational Benefits
- Predictable deployments
- Resource isolation
- Fast scaling
- Disaster recovery

Financial Industry Context
- Regulatory compliance through immutable infrastructure
- Reproducible model deployments
- Consistent backtest environments
- Isolated trading systems

Container Basics

What is a Container?
- Lightweight, standalone package
- Contains everything needed to run code
- Shares OS kernel but isolated processes
- Starts in seconds, minimal overhead

Key Benefits vs VMs
- Smaller footprint (MBs vs GBs)
- Faster startup time
- Better resource utilization
- Easier version control

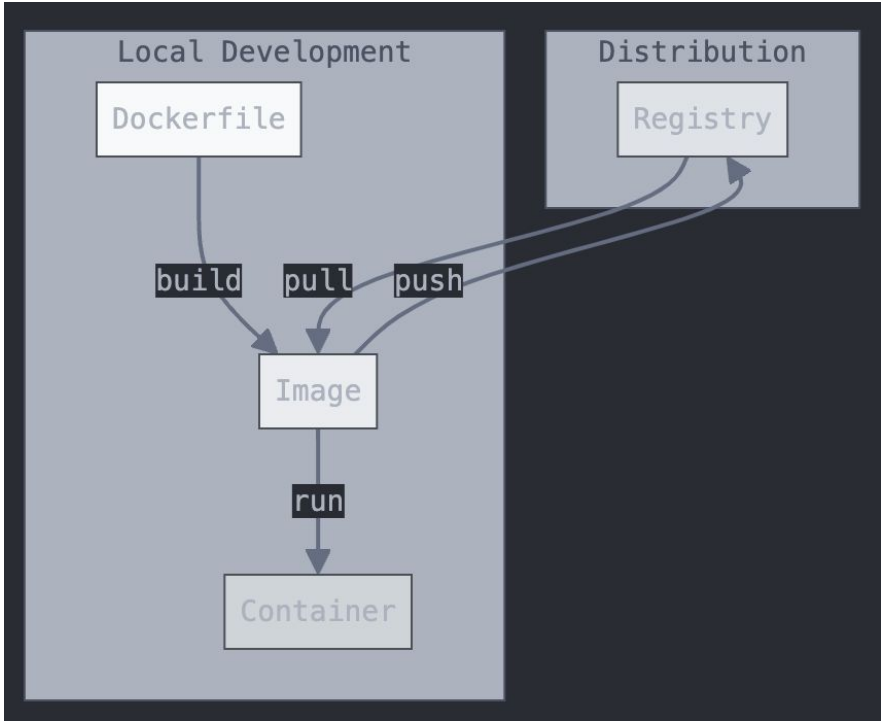


Docker Components

Images
- Blueprint for containers
- Layered file system
- Versioned with tags
- Immutable snapshots

Containers
- Running instance of image
- Isolated process space
- Defined resource limits
- Ephemeral by design

Registry
- Storage for images
- Version control
- Access management
- Distribution system



Dockerfile - Defining & Building Images

Key Components
- Base image selection
- Environment setup
- Dependency installation
- Application code
- Runtime configuration

Common Instructions
`FROM`: Base image
`WORKDIR`: Set working directory
`COPY`: Add local files
`RUN`: Execute commands
`CMD`: Default command
`ENV`: Environment variables

Layer Optimization
- Combine related commands
- Clean up after installations
- Use .dockerignore
- Minimize image size

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Name file `Dockerfile`

Place `Dockerfile` in the root directory of your project

Build image: `docker build -t <image_name> .`

Run container:

`docker run -p <host_port>:<container_port> <image_name>`

Stop container: `docker stop <container_id_or_name>`

Makefile

Why Makefiles?

- Command automation
- Build process standardization
- Team consistency
- CI/CD integration

How to use:

- Name of file: `Makefile`
  - `Makefile.dev`
  - `Makefile.prod`
- Place `Makefile` in the root directory of your project
- Call `make lint` instead of `black .`, `isort .`, `flake8 .`

```
lint:
    black .
    isort .
    flake8 .

test:
    pytest tests/

build:
    docker build -t model-service .

run:
    docker run -p 8000:8000 model-service
```

Final HW Assignment

Final HW

Serve your model through a containerized service

1. Create an inference endpoint with FastAPI
2. Write unit tests for your inference service
3. Lint your code
4. Add a makefile with commands that simplify linting and docker
5. Build an image for your FastAPI service with Docker
6. Run image locally and test that the inference endpoint works

Use Postman to test the endpoint locally or make a `curl` request through your terminal.

Common Repo Structure

/

- app.py
- tests
  - test_app.py
- Makefile
- Dockerfile
- requirements.txt